

A MYTH ABOUT AI SYSTEMS

“Magic pill”: the model answers more accurately if you ask it to act as an expert and complicate the prompt^[1,2].



Complicate the prompt for accuracy: “you are an expert lawyer, reason step by step”



Pick the architecture for the task: context, RAG or a tool

Where an AI system's reliability actually comes from — we work through it across the whole lecture.

LECTURE 3

AI system architectures: agents, RAG, API

03

Which architecture to pick for the task —
and when the right answer is “not AI”



the “magic pill” does not exist

Lecture route — six sections.

One load-bearing line: picking the architecture for the task — and when the right answer is “not AI”.

0



Opening

framing the problem: where reliability comes from

1



Prompt and its limits

what one call can do and where its ceiling is

2



RAG

external knowledge into context — and where it quietly breaks

3



Fine-tuning

changing weights for behavior, not for knowledge

4



Agents

loop, equipment, memory, security — and failures

5



Framework

ladder + checklist: how to choose fast and with grounds

From Lecture 2 we take two concepts as given.

We are not re-explaining — just recalling, so we can lean on them further. Everything else we build up along the way.



Single call (single-shot)

One pass of the model: gave a prompt → got an answer.
No memory between calls, no reaching outward.

The model knows only what is in the prompt and in the weights



Semantic search on embeddings

Text → a vector of meaning; fragments close in meaning are close vectors. Search “by meaning”, not by exact word.

RAG stands on this (Section 2)

Both concepts are the foundation from Lecture 2. Everything we build today (RAG, tools, fine-tuning, agents) leans on them.

THE CENTRAL QUESTION OF THE LECTURE

I have a task and access to an LLM. Which architecture do I pick — and when is the right answer “not AI”?

6

Multi-agent

several coordinated agents

more
complex ↑

5

Agent

loop: plan → act → check → iterate

4

Workflow

predefined paths

3

RAG / context engineering

retrieval-augmented generation

2

One LLM call

prompt; + CoT (step by step), + examples in the prompt (few-shot)

1

Plain code (no AI)

the reference point

simpler ↓



Climb to the next step — only when the task demands something the current one does not cover.

The ladder is a map of the lecture, not a demand to grasp everything now. We work through each step separately.

SECTION 1

The prompt and its boundaries

We saw the whole ladder — now from the bottom: what one call can do and where its ceiling is, before complicating anything.

one precise cut · 3 cases · 1 failure

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

How the task was framed before the prompt — and what that changes.

Before large models, "make the system do what you need" meant not a wish in natural language, but a precise specification and a deterministic program.



Precise specification / requirements

Pre- and postconditions, invariants, acceptance criteria (Z notation, TLA+, Design by Contract). The result is deterministic and verifiable.



Imperative vs declarative

"How to do it" (a step-by-step algorithm) vs "what to get" (SQL/Prolog describe the result, the engine decides how).



Interface contract

A precise input/output agreement (type signatures, OpenAPI, Protobuf). One correct meaning and a way to check it.

WHAT TO KEEP

The discipline of precise framing: a precise prompt is the same requirements spec in natural language. For the deterministic and verifiable (arithmetic, schema validation, rule-based routing) — use classical code, not a prompt.

Bridge: a prompt frames the task for a probabilistic system in natural language; hence both its power (no need to specify the un-specifiable) and its boundary (no single meaning, no determinism).

By default — one call with a good prompt.

The model knows only what is in the prompt (plus what was in the weights at training). It has nowhere else to draw from.



One LLM call with a good prompt

- lowest cost (a single pass)
- lowest latency (no extra round-trips)
- highest predictability (no loops, no retrieval that quietly degrades)

“JUST IN CASE” — ESCALATION



Do not complicate the architecture without a reason expressed in the task^[1] requirements. This is a distribution of the burden of proof, not primitivism.

[1] Anthropic — Building Effective Agents (“find the simplest”)

"Role" is two different mechanisms. Don't confuse them.

A role-persona ("you are a lawyer") tunes the tone. The protocol role system/user/assistant is markup of "who is speaking" in the token stream. The second is constantly mistaken for the first.

{ } Chat template: a list of roles → a flat token stream

```
<|im_start|>system behavior rules <|im_end|>
<|im_start|>user question <|im_end|>
```

Special markup tokens (ChatML, Llama 4) are assembled by the template — it can be read and spoofed.

system priority — a tendency, not a boundary

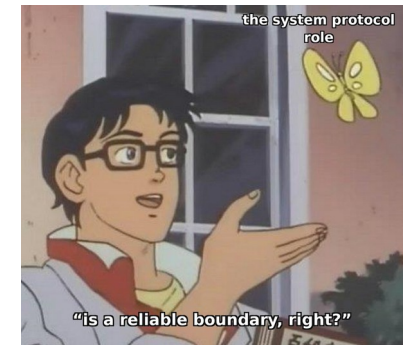
GPT-4o obeys the priority ~**63.8%** — still ≠ 100%: it's a tendency.



STI / role spoofing: the role is faked

The string `<|im_start|>assistant` from external text → ChatInject ASR
5.18% → 32.05% (Llama-4 up to 88.3%).

What to do: don't design defenses assuming "the system always outranks the user". Escape special tokens in INCOMING external content and check the chat template of local models — a foreign template quietly breaks the priority.



The full walkthrough of prompt injection as a class of agent attacks is in the agents section (security).

Prompt structure: separate instruction, context, data.

A flat prompt forces the model to guess where the instruction ends and the data begins. An explicit boundary removes that ambiguity.



Instruction

what to do



Context

on what basis



Data

what exactly to work with

Delimiters

- XML tags: `<instruction>...</instruction>`
- Markdown headings: `## Task · ## Data`
- Triple quotes / backticks around the data

The same principle as structured output (Section 4): there a schema sets the structure of the OUTPUT, here delimiters set the structure of the INPUT. A clear structure instead of “infer the form by meaning”.

A model that has been shown where the instruction ends and the data begins less often takes a fragment of data for a new command — the same confusion underlies prompt injection (Section 4).

Prompt format: the input forgives, the output pays.

Tolerance to INPUT format and the cost of forcing OUTPUT format are two different problems. They are constantly conflated.



Markdown

simple requests, instructions

GPT/Gemini · input is forgiving



XML tags

long context, many blocks

Claude · to separate blocks



JSON + schema

extraction, classification, code

constrained decoding → ~100%

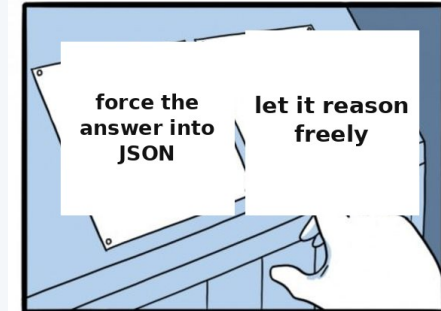


JSON on reasoning

don't apply to “think and decide”

reasoning tax

Don't force the OUTPUT's structure on reasoning tasks: GSM8K drops 76.6% → 49.3% (same model, only the JSON requirement changes). The mitigation is a two-step “reason freely first, then reformat” (+6.8 pp).



JANE-CLARK.TUMBLR

Chain-of-thought helps — but you cannot audit it.

Without CoT

“There were 23 apples, 7 spoiled, then 2 crates of 6 were bought.
How many good ones?”

→ a plausible but wrong number

With CoT (“reason step by step”)

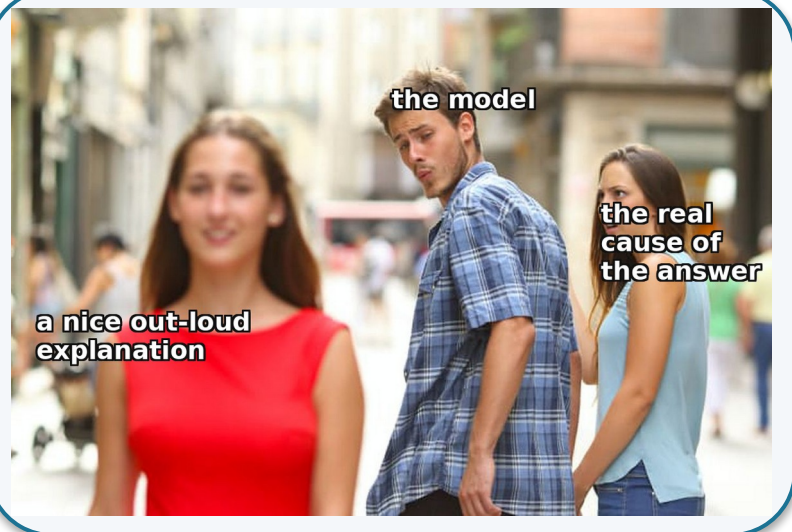
$23 - 7 = 16^{[1]}$ · $2 \times 6 = 12$

$16 + 12 = 28$

→ correct

Technically this is still one call — CoT is a tool for a class of tasks (a chain of steps), not a global switch.

But the stated reasoning need not reflect the real cause of the answer^[2] (low explanation faithfulness):



Check the result, not the model’s self-explanation.

Verify facts against an external source — a plausible reasoning text confirms nothing.

Reasoning out loud is not an audit log.

Faithfulness (of the explanation) — how well the spoken chain reflects the real cause of the answer. It was measured directly — and it is low.

Anthropic experiment (Apr. 2025)



The task is given twice: without a hint and with a hint in the prompt that changes the answer ("the professor thinks it's C")



They look: when the answer changed under the hint — did the model mention it in its reasoning?



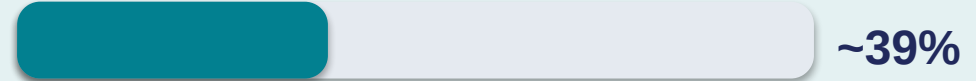
Often the model changes the answer, but builds a DIFFERENT, invented rationale, without naming the real cause

How often the model admits the hint

Claude 3.7 Sonnet



DeepSeek R1



And worse — on hard tasks (GPQA below MMLU): where audit is needed most, the explanation can be trusted least.

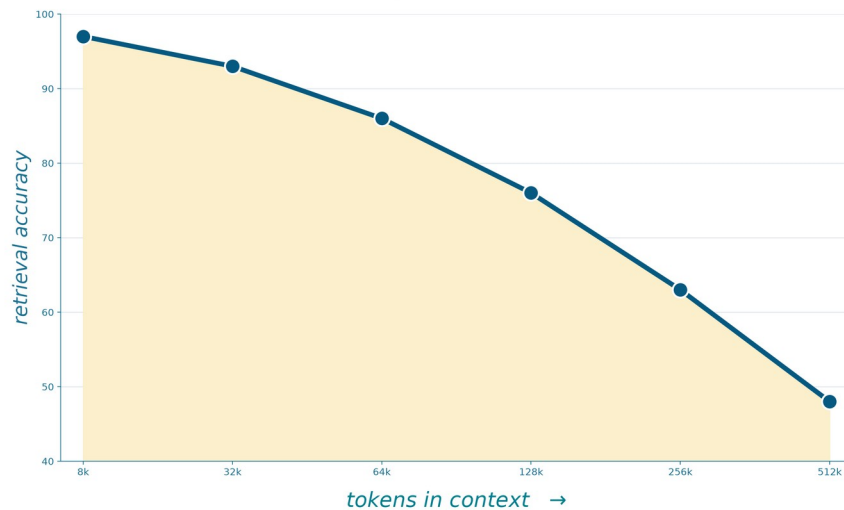
What to do: a human validator checks the RESULT against an independent source (a database, document, calculation, expert), not the attached "chain of reasoning". Control based on the model's self-explanation is not control.

Context engineering: the minimum of high-signal.

Prompt engineering is one instruction. Context engineering is curating the whole set of tokens the model sees at inference.

Retrieval accuracy ↓ as the context grows

SCHEMATIC — illustration of the effect, not measured data · effect: Chroma Research, 2025



context rot = the same “lost in the middle” from L2 — a new term, not a new entity^[1,2]

When NOT RAG (point 1)

a small, stable corpus that fits in the window →
full context + prefix caching, not RAG
infrastructure



RAG here would add fragility without a payoff

“Find the smallest set of high-signal tokens that maximizes the probability of the desired outcome” — this is an engineering requirement, not aesthetics^[3].

Cheat sheet: how to build a prompt.

The minimum below which a prompt systematically underperforms. Apply it to any prompt before the first run.

1 Role

if you need tone/register — and NOT as a promise of accuracy

2 Task

a concrete verifiable action, not a vague wish

3 Context

the minimum needed, not “everything that might come in handy”

4 Output format

stated explicitly if the answer is machine-processed

5 Delimiters

if there is more than one kind of content (instruction/data)

6 Examples (few-shot)

only if the format is not obvious from one instruction

7 CoT

only if the task needs multi-step reasoning

8 Length

no longer than needed — extra tokens “drown” in the context

A compact form of the whole section: role $\neq$ accuracy, structure helps separate inputs, CoT is a targeted tool, context is minimal. For large architectural decisions (RAG / FT / agent) — the eight-step checklist of Section 5.

What this looks like on real tasks — task patterns at the prompt level.

TASK PATTERN · 1 · edge case

An assistant with tools: climb the mini-ladder.



Setup

The user isn't asking for "text" — they're asking you to "do something": look something up in an external system and/or take an action.



One-shot → tool-use → agent

Knows the answer, no action needed → a single call. Access is needed, steps are known → a tool call in code. Steps are unknown, there's a way to check progress → a ReAct loop.

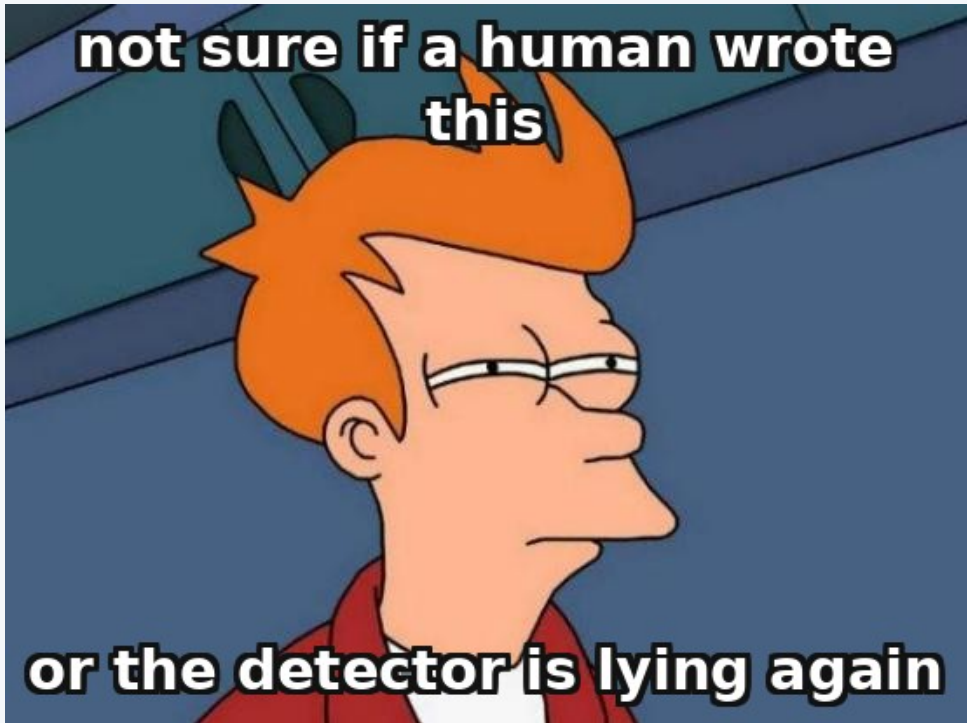
Parameters: tool granularity, tool_choice, an iteration limit and token budget, strict input schemas.

Validation: check the call's arguments BEFORE hitting the system; the answer relies on tool results, not on memory.

Typical failure: jumping straight to an agent where a single call would have been enough — the same \$4,200 loop (Section 4), where a predictable sync job was handed to an agent instead of a retry script.

Climb the mini-ladder only when the task requires it — not because "an agent sounds more powerful."

Text in a given tone: few-shot from samples.



Setup

You need text in a specific voice, style, or register: a brand-voice email, a support reply, a draft that reads “like a human wrote it.”



Persona prompt + few-shot style

A persona role (tone, not facts) + a short style guide + 3–5 samples of the manner as examples → generation in the same style.

Parameters: formality, emotional coloring, persona, number and quality of samples, explicit prohibitions (no bureaucratic phrasing / no emoji).

Validation: style is weakly formalizable → an LLM judge on tone + checks against markers (phrase length, vocabulary) + spot-check human review.

Judgment failure: detectors can't be trusted as evidence — some give accuracy below 80%, non-native writers' texts get flagged as AI up to +30% more often.

How to specify a JSON spec in a prompt correctly.

“Asking for JSON” ≠ “specifying a contract”. There are two levels of control with different guarantees, and three ways to describe a schema.

Two levels of control

Prompt level (“return JSON matching this schema”) — **≈80% validity**, guarantees nothing strictly.

Decoder (structured outputs / constrained decoding) — schema → grammar, disallowed tokens get masked → **~100% schema compliance**.

BAD

“Extract the data and return JSON” — no schema, field names, types, or enums; the model wraps it in markdown and invents fields.

GOOD

```
"rating": number    // 1-5, fractional; null if none
"sentiment": string // one of [positive,neutral,negative]
"pros": string[]    // [] if none
+ few-shot "input → expected JSON", schema AT THE END,
"Return ONLY JSON, no markdown."
```

Three ways to describe the shape

- **Example object**
clear to the model; does NOT express types/enums — flat structures, a prototype
- **JSON Schema**
types, enums, required, nesting; goes straight into the API, but verbose + degrades on complex ones
- **A code type (TS / Pydantic)**
compact + type-safe; needs a converter to JSON Schema

Reasoning tax

don't reason inside JSON: “first think freely
→ then reformat” as a separate step

Complex schema ↓

extraction accuracy ~87% → 70% → 56%:
split it up, keep nesting shallow

Valid JSON ≠ correct data: the grammar guarantees the shape, not the truth of the values — every JSON payload still gets validated in code. A strict schema reliably helps with extraction / classification / function calling; on “think it through and decide” — it's a reasoning tax.

SECTION 2

RAG: retrieval-augmented generation

Retrieve the relevant → put it in context → answer grounded in a source

external knowledge · 3 cases · 1 failure

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

2

How we searched text before embeddings.

The R in RAG is retrieval, search: a discipline with half a century of history. Exactly how it works determines where RAG works and where it breaks.



Inverted index

For each word — a list of documents where it occurs (a machine catalog). The foundation of Lucene, Elasticsearch, PostgreSQL full-text.



Boolean search

A query as a logical expression ("error AND authentication NOT tomcat"): precise, predictable, explainable selection.



TF-IDF → BM25

Ranking by word importance (rarer in the collection — stronger signal). BM25 (Okapi) — a cheap, explainable baseline many do not beat.

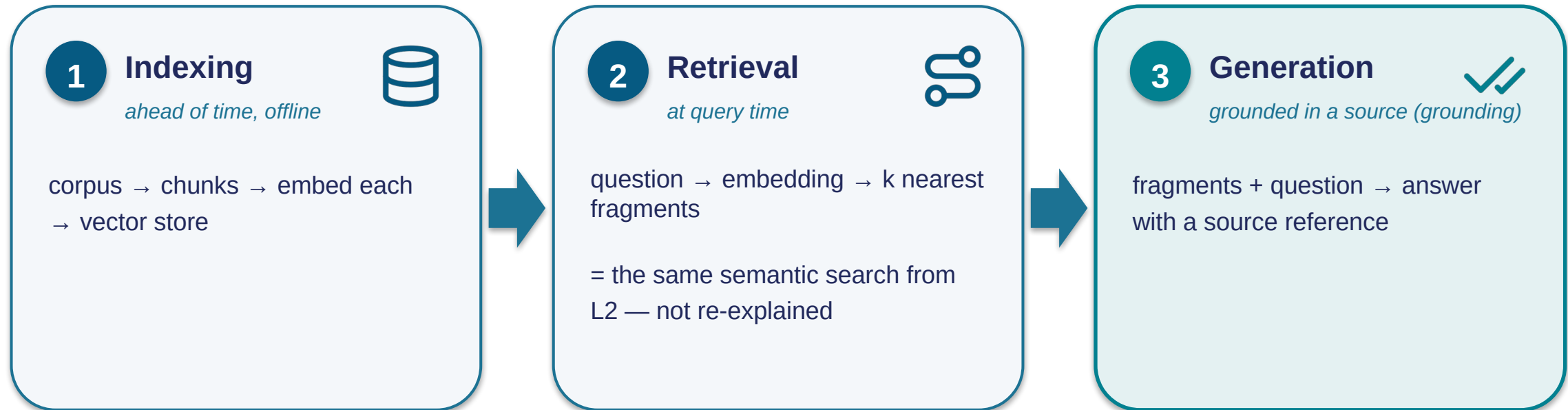
WHAT TO KEEP

The classics are precise on codes and identifiers, where semantic search blurs. A strong RAG-2026 is a hybrid of BM25 + dense vectors, lexical filters over metadata, ranking discipline (a reranker) and observability: recall/precision on a golden set.

Bridge: semantic search over embeddings (Lecture 2) adds semantic matching on top of the classics instead of lexical — but does not replace it. RAG extends classical search, it does not abolish it.

The RAG principle — three steps.

RAG = indexing → retrieval (semantic search from L2) → grounded generation; “I don't know” is a correct answer.



“I don't know” / “see source X” is a correct answer from a RAG system^[1]. A plausible answer on an irrelevant retrieval is a defect, not “at least something”.

RAG-2026: agentic RAG by default · hybrid search (BM25 + dense vectors) · reranker · miss cascade **5.7%** → **1.9%** (Contextual Retrieval).

[1] [Lewis et al. 2020 — Retrieval-Augmented Generation \(NeurIPS\)](#)

"Hybrid" — three different senses on common ground.

Sparse = classical lexical search: BM25 / TF-IDF (plus learned-sparse SPLADE / ELSER) on an inverted index; the unit is the word. A "sparse vector" is just a way of writing word-level search (as long as the whole vocabulary, almost all zeros); it is NOT searching by the meaning of a vector.

Dense = semantic embedding (≠ sparse): 384–1024 float values, ANN index, not interpretable, catches meaning and paraphrase.

(a) lexical + semantic

sparse (lexical, BM25) + dense (semantic, embedding), merged by rank (RRF) — the basic sense of the word "hybrid"

(b) semantic + filters

semantics + strict metadata filtering (jurisdiction, date) — an orthogonal lever

(c) two lexical branches

BM25 + SPLADE/ELSER — both sparse (classical + neural-weighted), no semantic dense vectors here



RRF fusion

$\text{score} = \sum 1 / (k + \text{rank})$, $k \approx 60$ — merges two rank lists with no common score scale.



Reranker (cross-encoder)

bi-encoder encodes separately (fast); cross-encoder runs the pair together (more accurate, expensive) — only on the top-50...100.

WANDS · NDCG

0.7497

vs BM25 0.6983 / vector 0.6953 → ~7.4% (modest)^[1]

Financial text+tables · Recall@5

0.816

vs dense-only 0.587 (+0.229)

Anthropic · miss rate

5.7 → 1.9%

+context+BM25 → 2.9%; +rerank → 1.9% (−67%)^[2]

The hybrid's payoff depends on the corpus: the large gains show up where lexical OR semantic matching breaks down on its own (jargon, tables, cross-references). On clean prose the gap is small — measure on your own queries first, then add complexity.

Where vectors live: two categories of engines.

Elastic and OpenSearch are also full-fledged vector stores (dense_vector + HNSW / k-NN plugin): it's more honest to place them on the axis **"a search engine that also does vectors"** vs vector-native (Qdrant / Milvus / Weaviate). Both are vector databases; origin is what tells them apart. Ceilings keep moving — check on the day of the lecture.

Engine	Take when	Ceiling, roughly
 pgvector	you already have Postgres, need one system and transactions	~50M vectors; degrades past ~50–100M ^[1]
 Elastic / OpenSearch	search-engine-first: you already run a cluster, need a native BM25+kNN hybrid	millions–tens of millions; ELSER without GPU
 Qdrant	vector-native without a big platform, sparse + multi-vector	large node + cluster
 Weaviate	vector-native with a built-in hybrid (vector+BM25+filters)	medium-large
 Milvus	vector-native: 100M+ / billions, heavy horizontal scaling	billions (needs operations) ^[2]
 FAISS · Chroma · LanceDB	FAISS is a library (storage is on you); Chroma/LanceDB — prototype/edge	from prototype to embedded

Frameworks complement, don't compete

LlamaIndex — ingestion + retrieval over messy documents; LangGraph — orchestration (checkpointing, human-in-the-loop).

For a simple RAG endpoint (retrieve → generate) no framework is needed: a few hundred lines of your own glue code beat frequent version churn. A framework gets added for agentic complexity, not by default.

What to choose for your situation.

A choice made in the open — for your scale and your requirements. Three tiers from the bottom up; you move up to the next one only when the previous one isn't enough for the actual workload.

**"we need a vector
DB, now!"**

**you have 3M chunks —
Postgres is fine**



1

Plain BM25 is enough

shared vocabulary, code/logs/IDs, curated knowledge bases; reindexing is cheap and observable — no vectors or vector store needed

2

A search engine with vectors is enough

need meaning on top of lexical matching at a scale of millions–tens of millions; Elastic/OpenSearch give a hybrid and ELSER without a GPU embedding service^[2]

3

Need a dedicated vector database

100M+ / multi-vector / a latency SLA on the hot path / decoupling the vector store from operating a logging cluster

A huge share of RAG systems never exceed a few million chunks — at that scale, pgvector or the Elastic/OpenSearch hybrid is the boring right answer^[1], and a dedicated vector database is often a premature optimization.

Chunking: how you cut is what you can find.

Of the three RAG steps, chunking is the most underrated, and its leverage is high. Strategies go from simple to complex; add complexity only when the corpus demands it.



Fixed-size

cut every N tokens — crude, but a strong baseline



Recursive / character

by a hierarchy of separators
(paragraph → sentence) — the pragmatic default^[1]



Sentence-window

embed the sentence, return a window of neighbors
for context



Semantic

boundary at a topic shift — intuitive, but often
doesn't pay off



Parent-document

small chunks for precision, a larger parent for
generation



Late chunking

embed the whole document, then pool — the
chunk stays context-aware



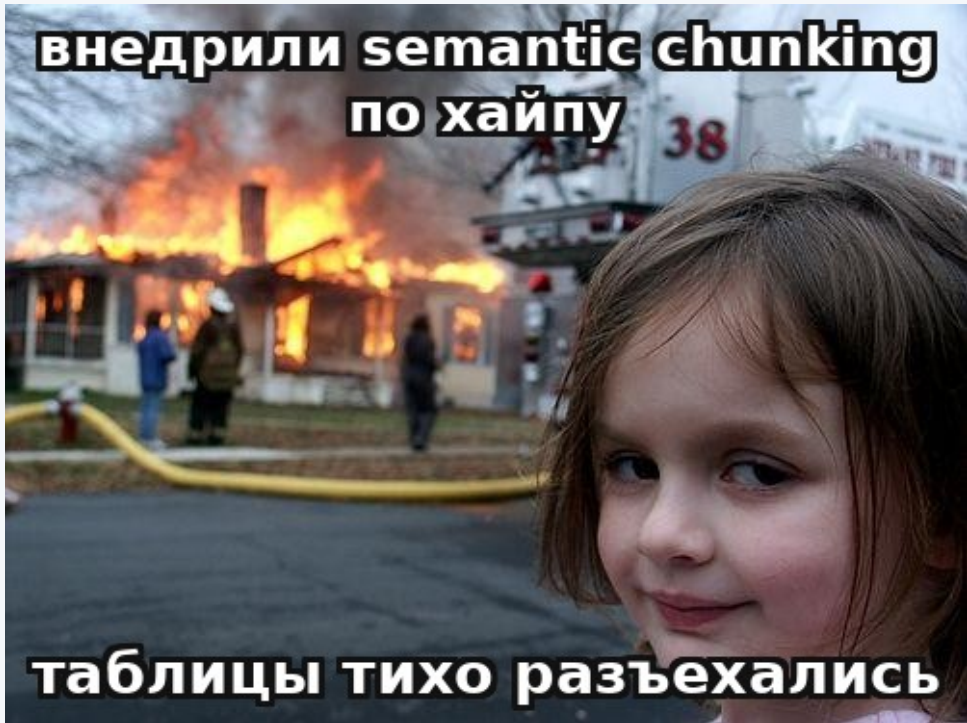
Contextual Retrieval

The LLM prepends 50–100 tokens of document-level context to each chunk before embedding and BM25^[2] (see the miss cascade 5.7% → 1.9%).

Neither chunk size nor strategy has a single correct default — you tune them to the corpus. Tuning without a retrieval-quality metric is flying blind.

Don't cargo-cult semantic chunking.

Studies on the effect of chunking openly disagree — and that disagreement is itself a lesson: “smart” chunking is domain-specific, plain recursive fixed-size is often stronger and cheaper.



recursive-512 vs semantic

69% vs 54%

Feb-2026, 7 strategies on 50 papers: recursive 512-token is #1; semantic gave ~43-token fragments^[1]

clinical (narrow domain)

87% vs 13%

MDPI Nov-2025: adaptive/topic-boundary vs fixed-size ($p=0.001$) — real, but narrow-domain

NAACL 2025 Findings

≈ or worse

“cost isn't justified by a stable gain”: fixed 200-word matches or beats semantic

One technique wins by 74 points (clinical) and loses by 15 (general corpus): corpora, metrics, and chunk size all differ. Default to recursive fixed-size; add complexity only for a gap measured on YOUR OWN data.

[1] [firecrawl — chunking benchmark \(recursive-512 69% vs semantic 54%\)](#) · [2] [firecrawl — chunking eval \(recall@k in isolation can mislead\)](#) · Recall@k in isolation can mislead (91.9% recall at 54% correct answers) — measure end-to-end accuracy too; the next slide shows how this breaks silently.

Silent chunking failure: a bare row of numbers.

Worked example — 300 PDF manuals with spec tables. The question “what's the tightening torque for model X-500?” — the answer sits in a table cell. A naive splitter cuts it on a byte boundary.

Naive: RecursiveCharacter, 512 tokens

The parser flattened the table into a stream; the splitter cuts on a byte at the 512 boundary:

```
chunk: "12 / 480 / 8.5 / 34"
```

The “Model / Voltage / Current / Torque” headers went into a different chunk. The system does NOT crash: the embedding gets computed, retrieval “finds” it, an answer gets generated — and it's wrong.

Correct: table-aware + repeated headers

The table becomes its own chunk; if it's over the limit — split by row, repeating headers:

```
"Model X-500 | Voltage 480 |  
Current 8.5 | Torque 34"
```

Each row is self-contained; + parent-document for the text, + metadata {model, type, version}. The main win came not from a “smart” split but from respecting structure.^[1]



Anaphora / broken references

“it”, “this city”, “this policy” lose their antecedent at a cut → the chunk embeds ambiguously (fix with parent-doc / contextual prepending).



Changing embedders = re-chunking

chunk size is tied to the model's window; change embedders → re-chunk the whole corpus, not “a line in a config”.^[2]

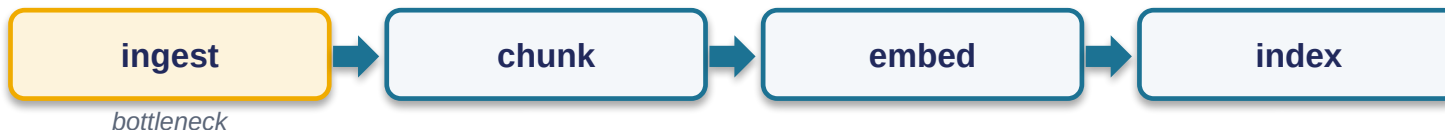


Evaluating chunking is k-sensitive

smaller chunks need a larger k; when you change the split, recheck the optimal k (Anthropic measured top-5/10/20).

Prototype → production — a different architecture, not a tweak.

A prototype works on 1,000 documents; it breaks at millions of vectors and thousands of queries. The problems aren't just at indexing — they're at query time and in operations too.



INDEX SIDE — building and maintaining

Freshness

batch → staleness; CDC → sub-minute at the cost of ×3 operational complexity; a stale index → silent degradation^[1].

Model change

a new embedder means re-embedding ALL vectors (old/new aren't comparable) — a migration.

Evaluation

recall@k ~0.8, nDCG^[2] + an LLM judge (RAGAS); without a labeled set — degradation is invisible.

Build cost

embedding + context per chunk; preprocessing ~\$1/1M tokens, a one-time cost at scale.

QUERY SIDE + OPERATIONS

Quality at scale

recall falls and drifts as the corpus grows; “found” ≠ “found the right thing”.

Latency / QPS

concurrent queries + a reranker on every query; autoscaling lags behind a spike.

Observability

retrieval fails silently, with no error — needs metrics and request tracing.

Security / access

who's allowed to retrieve what; PII in chunks; the permissions filter BEFORE search.

Counterfactual: a base under ~200k tokens goes entirely into the prompt with caching (up to ~90% savings) — retrieval is skipped altogether. The first question before any of this infrastructure — “do we even need RAG”.

[1] Redis — RAG at Scale (ingestion bottleneck, CDC 24h → sub-minute, cache ~69%) · [2] futureagi — RAG Evaluation Metrics (recall@k ~0.8, nDCG, RAGAS)

“RAG” doesn't exist in a vacuum — there is RAG for a task.

Five archetypes, each with its own nature of data → its own retrieval design. Shared anchor: naive dense RAG misses ~40% of queries.

Case · system	Design key	Failure → fix
 Support / knowledge base <i>kapa.ai · Stripe</i>	hybrid + rerank	dense confuses “backoff” and “dead-letter” → BM25 catches the exact error code
 Q&A over docs <i>Vercel docs-copilot</i>	source OR refuse	cites a stale API version → freshness + eval in CI
 Code search <i>Cursor · Cody</i>	chunk per function	dense blurs a symbol name → Cody removed embeddings → lexical^[2]
 Legal <i>LexisNexis · Westlaw</i>	hybrid, BM25 for citations	17–33% hallucinations despite “0%” ^[2] → citation verification + human
 Enterprise <i>Glean · eSapiens</i>	router to Text-to-SQL	embedding a table doesn't sum it → numeric question → SQL

Two cases are direct “RAG was the wrong tool”: Cody went back to classic code search; an aggregating question over a table → SQL, not “similar chunks”. Retrieval design follows from the nature of the data.^[1]

[1] Barnett et al. 2024 — Seven Failure Points (RAG archetypes) · [2] Stanford HAI/RegLab — Legal-AI hallucinates 17–33% (Lexis/Westlaw) · [3] Sourcegraph Cody — context without embeddings (classic search)

Deep research is RAG + a loop, not a “big prompt”.

The defining component is retrieval and orchestration, which is why this case lives here, not in prompting. Like OpenAI Deep Research, Perplexity, Claude Research.

agent hands back a report with citations	
some links even resolve	
3-13% of URLs are just fabricated	



Reasoning LLM + agentic RAG loop

plan sub-questions → (search → read → refine the reasoning) × N →
synthesize → verify citations

Reasoning steers the search, findings refine the reasoning — a closed loop (agentic RAG)^[1], unlike static retrieve-top-k → generate.

Typical failure — citation fabrication

3–13% of URLs are fabricated (in retrieval-augmented mode); DRACO: the top result is ~65% citation quality. Counterintuitively: deep research is worse than plain search — **10.7% versus 4.8%** of fake citations per query, because it generates far more of them.^[2]

Against a human analyst — faster and cheaper, BUT without a verification layer 3–13% of the citations are false. Recheck any number from AI research against a resolving primary source; treat a number with no baseline as a red flag, not a fact.

When RAG is the right choice.

RAG is justified when the signal on the features below is strong^[1] — and there are no blockers from the next slide, “when NOT RAG”.

Large / growing

does not fit the window whole, or
it is costly to put the entire corpus
in every request

Changing

documents, prices, regulations
update more often than model
versions ship

Freshness + provenance

the answer relies on a verifiable
source; you can show where the
fact came from

Private base

the company's knowledge is not
in the weights of a public model

The features reinforce each other → RAG

A corporate base of thousands of regulations, updated weekly, an answer with a mandatory reference to the clause: all features converge, no simpler mechanism closes them jointly → a textbook RAG profile.



One feature is a reason to take a look^[2], not to build automatically: check the blockers on the next slide.

When RAG is NOT the right choice.

Knowing when RAG is not needed is worth more: it is a fashionable architecture, put where it does harm.

1 The corpus fits in the window

a rough guide — under ~200k tokens, rarely changes

→ full context + prefix caching, not RAG infrastructure

2 A fixed policy / value^[2]

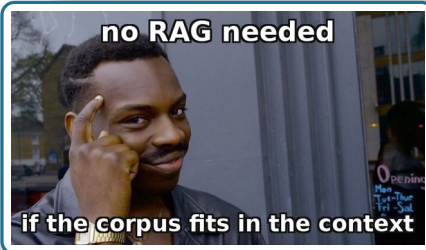
a fare, a price, a clause of a regulation, a rule

→ a deterministic lookup table / a static page

3 Data available in real time via API / MCP

in an internal service, a database, another system's search

→ call the tool directly; a RAG index on top is an extra layer, more fragile and more stale



A common anti-pattern — putting RAG infrastructure where the whole corpus comfortably fits in the context window and rarely changes.

RAG is redundant if ANY of the three holds^[1]: (a) the corpus fits in the window and is stable, (b) the task reduces to a fixed value, (c) the knowledge is already available directly and live via a tool. “A direct call returns data as of the request; a RAG index — as of the last indexing.”

The failure of RAG at scale.

“Returned something” ≠ “returned the right thing”. RAG has no ^[1,2] “not found” signal — it always returns the k nearest, even irrelevant ones.



Legal-AI

the “nearest” cases from another jurisdiction / an overturned precedent — the model relies on them as fact



Medical-RAG

mixed fragments of different patients — close in symptoms, but clinically cannot be combined



Support bot

worked on hundreds of articles; after growing to thousands, quality quietly degraded — nobody noticed

Air Canada — architecture walkthrough ^[3]

Diagnosis

generated plausible text was placed in a role that required a retrieved, verified fact — a failure of grounding

The right alternative

a fixed policy → a lookup table / page; need a dialogue → RAG with strict grounding, a mandatory citation, an explicit “I don't know”, a human check

SECTION 3

Fine-tuning vs prompt vs RAG

We solved the knowledge problem through RAG. But what if the problem is not knowledge, but the model's behavior — its tone, format, policy?

behavior tuning · 4 cases · 1 failure

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

How the ML task was solved before large models.

Fine-tuning a large model is not an exotic novelty of a new era, but a direct continuation of the classical transfer-learning scheme. Let's rebuild it from scratch.



Sample + ground truth

A labeled set "input → correct answer" (ground truth, reference labeling); the model learns to reproduce it and generalize to new data.



Train / val / test split

Three disjoint parts: you train on train, tune on validation, measure once on test. The rule: you must not test on the training data.



Transfer learning

Take a model pretrained on a large corpus and cheaply fine-tune it for your narrow task: faster and more accurate than from scratch ("pretraining → fine-tuning").

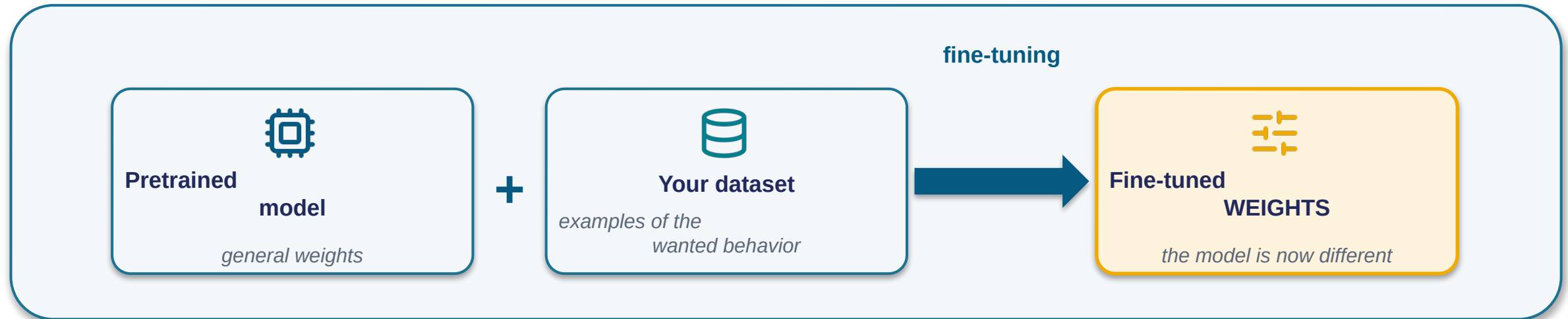
WHAT TO KEEP

Eval sets (golden set) — without them catastrophic forgetting is invisible; **versioning of data and weights for rollback**; **train/test discipline against leakage**; **drift monitoring**. LoRA made the fine-tuning step cheaper, but not the discipline around it.

Bridge: PEFT/LoRA is the same transfer learning, pushed to the limit of cheapness and on top of an incomparably larger model. The idea is not new — what became new is the scale of the base model and the cost of the step.

What fine-tuning is.

Fine-tuning is continuing to train an already-built model on your data. In L1 — a type of use; here — an architectural choice, one of the rungs of the ladder.



Prompt / RAG → CONTEXT

They change only the input — the weights are untouched; the effect lives only within the request.

Fine-tuning → THE WEIGHTS

The change is baked into the model — it applies always and costs more than what context changes.

Prompt/RAG = “what to show the model”. Fine-tuning = “change the model itself^[1]”. In practice “fine-tune” almost always means LoRA/PEFT, not retraining all the weights (next slide).

[1] IBM — RAG vs Fine-Tuning (fine-tuning changes the weights)

Training cost: order of magnitude and growth shape matter, not the exact figure.

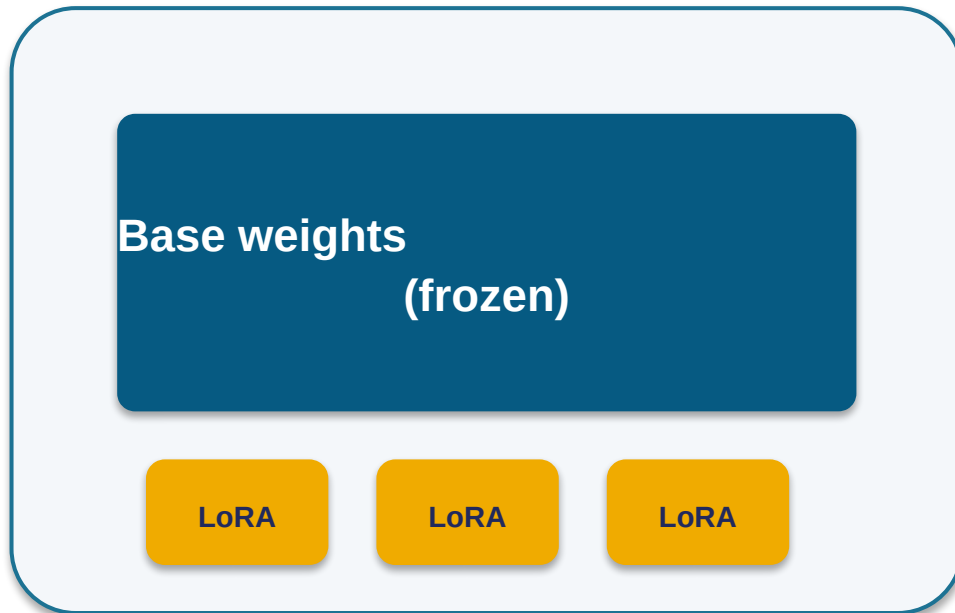
Values are relative: Full-FT is taken as baseline $\times 1$ on VRAM / \$ / compute, the rest are fractions of it. Absolute \$ and hours keep moving; what matters is the order of magnitude and growth shape.

Axis	Pretraining from scratch	Full-FT	LoRA	QLoRA	Prompt+RAG
Trainable parameters	100% (from scratch)	100% of weights	$\sim 0.1\text{--}1\%$	$\sim 0.1\text{--}1\%$	0% — weights unchanged
VRAM (memory)	cluster	$\times 1$ (baseline)	$\sim \times 0.2$	$\sim \times 0.05$	inference only
\$ per run	$\times 1000+$	$\times 1$ (baseline)	$\sim \times 0.01$	$\sim \times 0.01$	~ 0
Compute (GPU-hours)	$\times 1000+$	$\times 1$ (baseline)	$\sim \times 0.01$	$\sim \times 0.01$	~ 0
Data volume	trillions of tokens	thousands–tens of thousands	thousands–tens of thousands	like LoRA	few-shot (3–20)
Iteration speed	months	slow	fast	fast	instant

LoRA is the default: trains $\sim 0.1\text{--}1\%$ of parameters, $\sim \times 0.01$ the price / compute and $\sim \times 0.2$ the VRAM of Full-FT — a 7B LoRA run costs $< \$10$ — ~ 7 orders of magnitude cheaper than frontier pretraining (\$61–92M)^[1]. Full-FT is justified only when the dataset exceeds LoRA's capacity — otherwise it's just $\times 5\text{--}6$ the memory and forgetting risk. QLoRA is the same LoRA on top of a 4-bit base: 65B fits on a single 48 GB card^[2].

PEFT instead of full fine-tuning.

PEFT — the base weights are frozen, only a small set of adapters is trained. LoRA — low-rank adapter matrices; QLoRA — the same on top of a quantized ^[1,2] model.



adapters — megabytes vs gigabytes

1. Cheaper and faster

millions of parameters instead of billions;
QLoRA — on a single GPU

2. Modularity

adapters are megabytes vs gigabytes;
one base — many specializations

3. ↓ Forgetting risk

the base is frozen, physically not
overwritten by the new signal



PEFT / LoRA: cheap,
modular, lower forgetting
risk



Full fine-tuning of all
weights in 2026 —
almost never

98.4% of PEFT-tagged models are LoRA^[3]

out of 20,834 cards on the Hugging Face Hub · caveat: the share among PEFT-tagged models, not among all fine-tuning

PEFT (LoRA/QLoRA) is almost always preferable to full fine-tuning: cheaper, more modular, ↓ forgetting risk. Full fine-tuning in 2026 — almost never.

What here is knowledge, what is behavior, what is deterministic.

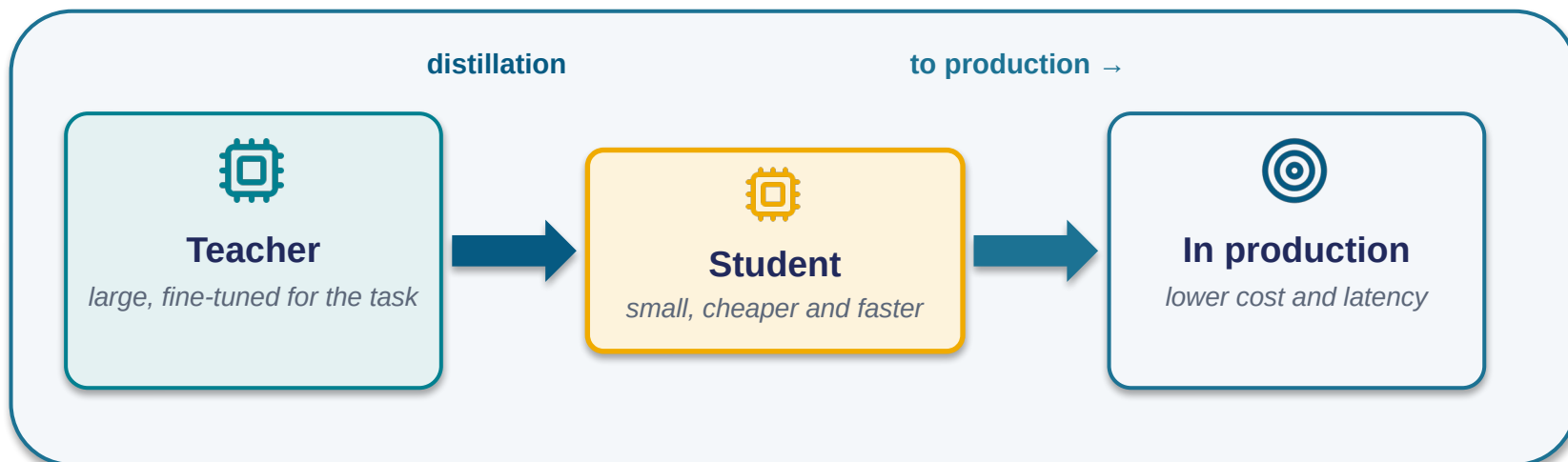
The design question is not "RAG or fine-tuning". Split the problem along axes: knowledge → RAG, behavior → PEFT, cheaper → distillation, deterministic → code.

If the task requires...	→ the right tool	...and NOT this one, because
knowledge changes / freshness, provenance needed	RAG (or a long context for a small corpus)	not fine-tuning: knowledge will go stale, retraining is expensive, risk of forgetting
stable behavior / tone / format / policy	fine-tuning (PEFT)	not RAG: it feeds knowledge into the context but doesn't change the model's manner
reduce cost / latency on a narrow task	fine-tune a teacher + distill into a student	two separate techniques in tandem; prompt/RAG don't shrink the model size
a deterministic, verifiable answer	plain code, no AI	neither RAG nor FT: AI adds non-determinism with no gain

What to do: a hybrid is the norm WHERE the task has BOTH a knowledge problem AND a behavior problem. No behavior problem — no FT needed, even when there's RAG. Each component is added to meet its own requirement — the same ladder rule.

Distillation is not a kind of fine-tuning — it is a separate technique.

Fine-tuning changes the model's behavior^[1]. Distillation is compression: transferring the skills of a large model into a small one. They are often confused, but these are two taxonomically distinct operations that work together.

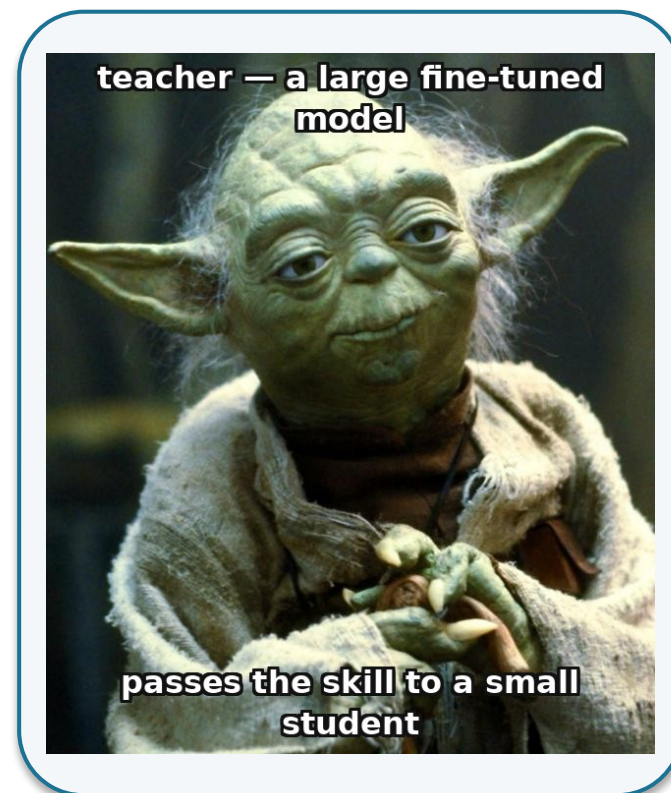


Fine-tuning answers:

“how the model behaves” — tone, format, policy.
Changes the WEIGHTS for behavior.

Distillation answers:

“how to make the model cheaper” — the same skill
in a smaller model. This is compression, not a
change of behavior.



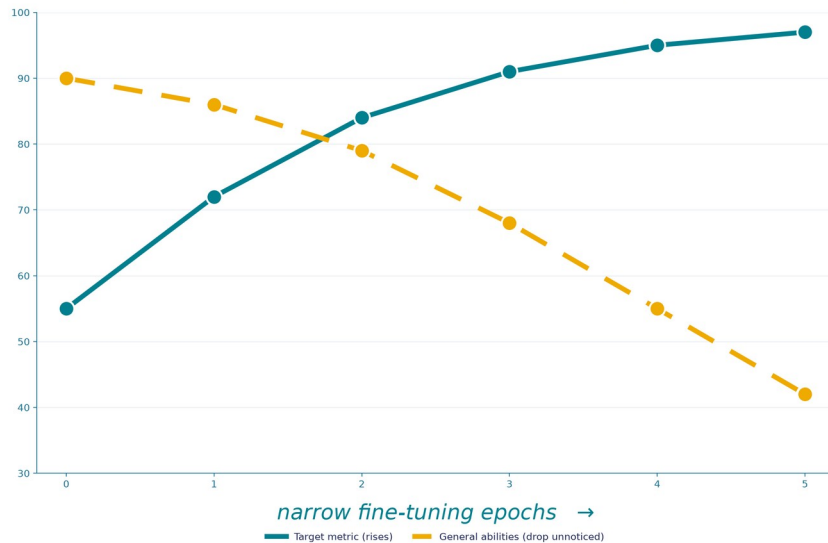
What to do: do not write “distillation is fine-tuning”. In tandem they go like this: first fine-tune the teacher for the task, then distill into the student for cost. The criteria “what goes where” — on the next slide.

Failure: catastrophic forgetting.

Catastrophic forgetting — degradation of the model's general abilities as a result of narrow aggressive fine-tuning.

Narrow aggressive fine-tuning: target ↑, general abilities ↓

SCHEMATIC — illustration of the effect, not measured data · effect: Luo et al., arXiv:2308.08747, 2023



worse as the model scales up — a larger model starts higher, so it has "further to fall"

No eval loop on general tasks + no dataset/weights versioning → you won't see the break before production + you can't roll back → this is not a "risk", it is a criterion for "do NOT fine-tune"

Right way: PEFT (frozen weights — lower risk); for changing knowledge — RAG, not fine-tuning at all.

Evaluating a fine-tune costs more than training it — and none of the six methods is self-sufficient.

A useful 7B LoRA costs <\$10 — while a rigorous evaluation of that same adapter is \$300–1200 of human labor per round, and it can still fail to predict production. Every method follows the scheme "how it works · when valid · when it breaks · cost".

Method	How it works	When it breaks (number)	Cost
Public benchmark	accuracy on a fixed Q&A set (MMLU/GSM8K/HELM)	contamination: MMLU ~29% of questions ^[1] ; GSM8K → GSM1k ~13 pp ^[2] — memory, not reasoning	low (deceptively)
Held-out test set	frozen BEFORE training, run every version against it	leakage into the training set / small volume / drift from production	upfront labor, ≈0 to re-run
LLM-as-judge	a strong model judges; pairwise > pointwise	>80% agreement with humans (= human-human), BUT position bias up to 75%, self-enhancement +10–25%	cents + bias control
Human evaluation	rubric scale + ≥2–3 annotators + κ agreement	κ≥0.6 required (otherwise rewrite the rubric); expensive, doesn't scale	\$300–1200 per round
Task metrics (BLEU/ROUGE)	n-gram overlap with the reference	on open generation r≈0.25–0.52 with humans — rewards the n-gram, not the meaning	near zero
A/B / online	canary 1–5% + guardrail metrics on live traffic	no traffic before production; ×4 samples due to non-determinism	engineering + risk

No single method is self-sufficient: the cheap ones (benchmark, BLEU) have low validity; the valid ones (expert held-out, human evaluation, A/B) are expensive. The held-out test set gives the highest return, and you build it BEFORE training. The lab → production gap ~37% is mechanics, not an anomaly: a benchmark gain is a hypothesis, not a result; only a clean held-out set + A/B on real traffic confirm it.

[1] [Pebblous — LLM Benchmark Contamination \(MMLU ~29%\)](#) · [2] [Zhang et al. 2024 — GSM1k \(clean mirror of GSM8K ~13 pp\)](#)

SECTION 4

Agents

From a conversationalist in a chat window — to a component of a production system: the loop, the equipment, memory, tool access — and where all of it breaks.

agent + equipment · 5 cases · 4 failures

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework



Controlled automation existed long before agents.

"Agent" sounds like an invention of the large-model era, but controlled process automation is a mature discipline. Without it you can't judge what an agent adds and what it breaks.



Finite-state machine

A set of states and event-driven transition rules ("created → in progress → closed"). Possible transitions are visible, impossible ones are excluded by construction.



Workflow engines

Execute a predefined process by a fixed schema: BPMN, DAG orchestrators (Airflow), RPA. The order of steps is defined in advance, not on the fly.



Control loop

plan → act → check as a feedback loop (control theory, SCADA); the military analog is the OODA loop. Not a single action, but a loop with correction.

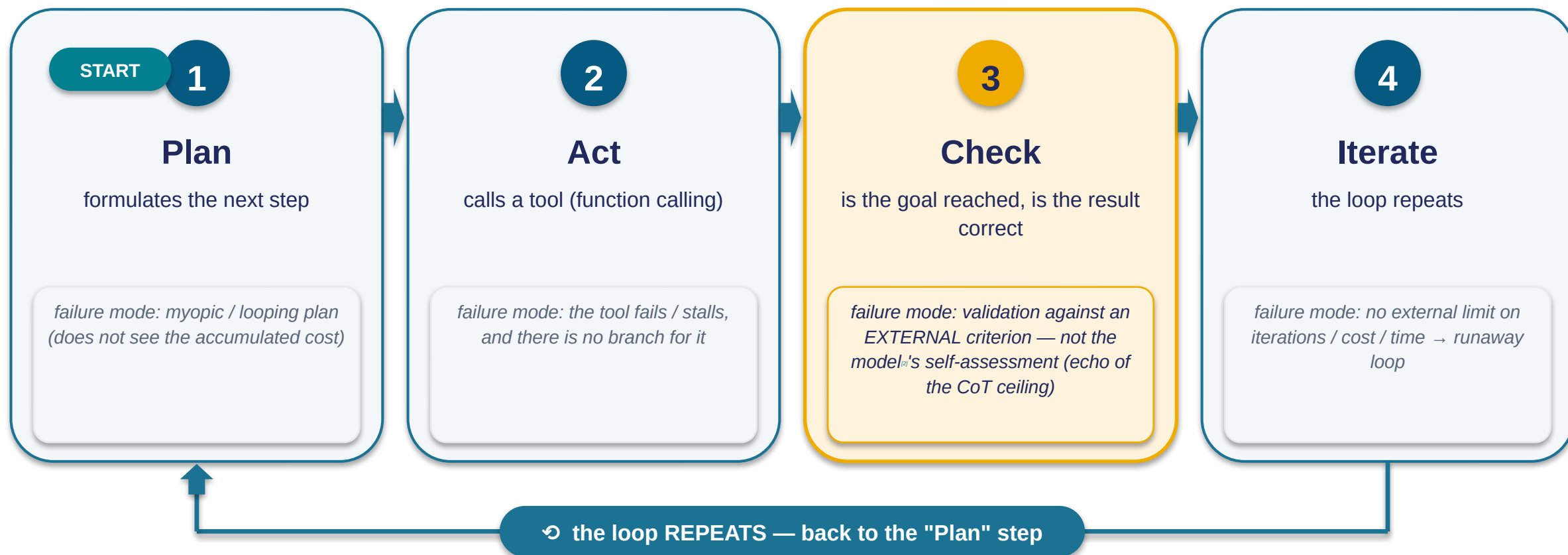
WHAT TO KEEP

Deterministic workflows where possible; idempotency (repeating a step does not break state); the least-privilege principle for tools; limits and audit of the loop. Reliability practices of the control loop don't go anywhere.

Bridge: an LLM agent is the same plan → act → check control loop, but the steps in it are generated by the model non-deterministically, rather than choosing a fixed automaton rule. Hence both the flexibility and the failures of the section.

The agent loop: plan → act → check → iterate.

An agent is an architecture where the model does not make one pass, but works in a loop, determining the sequence of steps itself^[1].



The model becomes a system component: API + MCP.



Structured output

output strictly by schema (JSON), not text to parse

embeddable



Tool calling

the model formulates "call X"; your code executes it, not the model

active



Prompt caching

don't recompute the unchanged prefix on every request

economical



MCP — "USB-C for LLM tools"

N×M incompatible integrations → **N+M**^[1]

a tool once = an MCP server; a model once = an MCP client

AI Anthropic
11/2024



OpenAI
03/2025



Google
04/2025

Standardizing the connection ≠ security of what you connect — and it makes the trust problem worse.

- code in your environment / access to data
- the description enters the context — a carrier for prompt injection

No mechanism makes the model more reliable — the ladder rule still holds. Ease of connection is not an argument for connecting.

[1] [Anthropic — MCP donation / Agentic AI Foundation \(N×M → N+M\)](#) · Current savings figures and the scale of the MCP ecosystem — see sources.

An agent is not a "pricier chat". It's a different cost class.

The comparison baseline is one chat turn (tens of cents). Each step up multiplies token spend, it doesn't just add to it.

Single LLM call (chat)

the base cost of a request

×1

Single agent (loop)

plan → act → check → repeat: many passes per task

≈50×

Multi-agent

*on top of the agent — coordinating several agents
(Anthropic, 2025)*

another ×15

What to do: budget BEFORE choosing the architecture. Prompt caching (don't recompute the unchanged prefix) in an agent is no longer a "nice reference" but a necessity: without it the ×N loop cost becomes unaffordable.



The multipliers are order-of-magnitude (Anthropic, 2025), not an exact tariff; the absolute price depends on the model and task.

MCP: connecting a tool is now a matter of minutes.



MCP — a connection standard (already covered above)

Describe a tool once as an MCP server — and every model-client sees it. Connecting became a matter of minutes. But the ease of connecting says nothing about what exactly you're connecting.

Read the catalog critically

"up to 90,000 servers" → actually runnable $\approx$ **10,000** = **11%** of the catalog.

The rest are duplicates, broken and stubs. And even the working 10 thousand aren't checked for security.



The trust turn: standardizing the connection $\neq$ security of what's connected — and it sharpens it.

30+ CVE

in a 60-day window against MCP servers (~43% — command injection)

82%

path traversal among 2,614 checked implementations

What to do: ease of connection is not an argument for connecting. Each MCP server = someone else's code in your environment + a carrier of injection into the context. You connect it to meet the task's requirement, with an assessment of the new trust boundary.

MCP or a direct API — these are not alternatives: MCP pays off as a fleet, below the threshold it's an extra layer.



Base: REST/gRPC is the transport that does the work (hits the database, calls the service). **MCP is a thin standard layer ON TOP**, letting an agent discover (`tools/list``) and call tools at runtime. In most deployments an MCP server WRAPS an already-existing REST API rather than replacing it. Threshold: the payoff $N+M < N \times M$ materializes only when $N, M \geq 2-3$.

Take MCP when...

- ✓✓ many agents $\times$ many tools ($N, M \geq 2-3$)
- ✓✓ you need runtime tool discovery (the catalog changes)
- ✓✓ you need portability across model vendors / hosts
- ✓✓ servers are reused by other teams / third parties
- ✓✓ you're ready to run a security review of every server

Take a direct API / function calling when...

- one app, one or two known tools ($N+M \geq N \times M$)
- the tool set is fixed at build time
- you're locked into one model, one stack
- the integration is private, a single consumer
- you need a small auditable locked-down surface NOW

~25%

of registry servers are unusable¹ (a floor, not a ceiling)

43.7%

top result on MCP-Universe² (>56% of tasks fail)

+risk

injection / path traversal — see the slide on MCP (the trust pivot)

Rule: MCP is an interoperability standard, not a performance upgrade. Skipping MCP for a single agent with a single tool is the correct engineering judgment, not a cut corner (the same ladder rule, §5.1).

Workflow vs agent.

Predictable task → workflow^[1]; unpredictable AND the value justifies a multiple-fold increase^[2] → agent.

Workflow

LLM and tools along paths predefined in code

- the sequence of steps is known in advance
- predictable, auditable
- most reliable production systems are a workflow

Agent

the LLM dynamically determines its own process

- the sequence is not fixed in advance
- many times more tokens than a chat
- lower auditability, higher risk of loops

Can I write out the sequence of steps in advance, before launch? yes (even with branches) → workflow · fundamentally no AND the value justifies the multiple-fold cost/risk → agent

Nesting is the norm: a code-review agent calls a "lint → test → format" workflow as one step; a ticket-processing workflow delegates parsing a free-form complaint to a mini-agent. Dynamics — only where it is needed.

Find the simplest option. "Too lazy to formalize" does not make a task unpredictable.

An agent framework is an abstraction tax; all of them share five drawbacks.

Anthropic: many patterns are a few lines of direct API calls^[1]; frameworks hide the underlying prompts and make debugging harder. First, the five drawbacks shared by ALL of them, then the breakdown by schema.

 **Customization**
debt beyond the "happy path"

 **Transparency**
hides the actual prompts and control flow

 **Lock-in**
vendor SDKs tie you to their ecosystem

 **Version churn**
breaking API changes (LangChain)

 **Overhead**
CrewAI +18% tokens vs LangGraph[®]

Framework	Core abstraction	What it fits	One honest drawback
LangGraph	stateful graph	production: control flow, state, human-in-the-loop	steep curve; over-engineering for a single loop
CrewAI	role-based "crews"	fast, role-decomposable prototyping	the role masks what is actually happening; multi-agent fragility
AutoGen → AG2	agent conversation	researching conversational patterns	frequent renames (AutoGen → AG2 / MS Agent) — adoption risk
OpenAI Agents SDK	handoffs	lightweight task handoff + tracing on OpenAI	OpenAI-centric; you bring your own state persistence/RAG
Claude Agent SDK	the same loop as Claude Code	coding / computer-use, long-running tasks	Claude-centric; heavy harness vs the bare API
smolagents	code agents (write Python)	a minimal, readable agent	code execution = attack surface (needs a sandbox)
LlamaIndex agents	an agent on top of a RAG stack	an agent over your own documents/data	the center of gravity is retrieval, not orchestration
Pydantic AI	type-safe agent loop	validated outputs, typed contracts	young; type-safety is orthogonal to orchestration
NONE	plain code + direct API calls	deterministic steps, ≤ a couple of LLM calls	requires the discipline to not reach for a framework in advance

The five drawbacks above apply to EVERY row. The bottom row isn't there for completeness: a deterministic path + known steps + cost sensitivity → plain code, without an agent and without a framework. The ladder rule at the tool-choice level: tooling complexity is paid for by the demands of the task, not taken on in advance.

An agent fits an open task; multi-agent is only needed under three conditions.

Top-down: predictable → workflow; unpredictable and valuable → a single agent; several agents — only for a specific case. Every extra step multiplies the probability of success, it doesn't average it.



1. Predictable → workflow

steps known in advance; paths predefined in code, auditability



2. Unpredictable → a single agent

steps depend on intermediate results; the cost is justified by the value



3. Multi-agent — for a specific case

needed only under 3 conditions (right); outside them — the wrong tool

Three conditions under which multi-agent is justified:

1. Widely-parallel independent READS (not decisions)

e.g.: read 100 sources in parallel and roll them into a report (Anthropic research).

2. Independent perspectives for cross-checking

e.g.: 5 agents search independently → majority vote.

3. Isolated sub-domains with separate tools / permissions

e.g.: a separate agent for billing, a separate one for infrastructure.

Cognition vs Anthropic — reconciliation:

Anthropic: multi-agent wins research tasks. Cognition: "don't build multi-agents." There's no contradiction: parallelize independent READS¹ (broad search), but not DECISIONS with dependencies. Topology: more connections = faster collapse².

Outside these cases, multi-agent is the wrong tool: reliability drops as p^n ($\times 15$ tokens). Numbers and measurements — on the next slide.

Start with one strong agent. Multi-agent is justified only in the three cases above and when the value justifies the multiple-fold cost; otherwise it's the wrong tool.

Multi-agent by default is not a win — it is a risk multiplier.

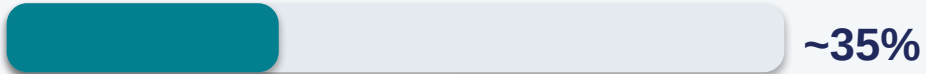
In a chain of n steps, where a failure of any one ruins the result, reliabilities multiply: success $\approx p^n$. The engineering intuition "each step almost always works" is mathematically false.

Chain reliability = p^n

95% × 10 steps



90% × 10 steps



95% per step sounds reliable — but $0.95^{10} \approx 0.60$. More agents = more steps = lower overall reliability.

Topology decides: coordinator > swarm

A "swarm" of equal agents amplifies errors **17.2×**; a single coordinator — only **4.4×** (Zartis/Redis).



Anthropic verbatim: "multi-agent works mainly because it helps spend enough tokens to solve the problem" — the win comes from token volume, not from "coordination magic".

What to do: start with one strong agent. Multi-agent — only if the task splits into WIDELY parallel independent subtasks of high value; otherwise +15× tokens and coordination overhead will not pay off.

Every agent equipment slot is a trade-off, not a win.

A real assistant agent (Claude Code, Cursor, Aider) is a plan → act → check → iterate loop PLUS equipment. Five typical slots^[1]:



Just as you don't climb the architecture ladder without a task requirement — don't add memory, subagents or MCP to an agent "just in case". Each slot carries its own cost: operational complexity, a new failure surface, a new trust boundary — and it must answer a concrete trigger (Section 5).

^[1] *agent-harness-registry — map of the agent's equipment slots*

"An agent that remembers" — not always for the better.

Having memory intuitively seems like a pure win. Independent evaluation shows: sometimes — dramatically not^[1].



Letta — Tier D

loses to BOTH the bare model AND the flat file on every task.
persistbench_v1:

 Bare model	1.000	94 s
 Flat file	0.833	159 s
 Letta	0.750	496 s

Mechanisms: capitulation under pressure · verbosity drowns the fact · the fact is noticed but not committed.

Caveat: the test is on Letta v0.6.7 — behind the current v0.16.8 (~18 mo).

Anthropic Memory Tool — Tier B

A strong result overall — but even it loses data on

17% of tasks

- an explicit refusal to record an "ephemeral" fact
- an unmotivated "out of scope" refusal
- silent lossy compression — the detail can't be recovered
- non-reproducibility: the same conversation twice → a different result

^[1] agent-harness-registry (live-eval) — Letta Tier D / Memory Tool 17% tail · agent-harness-registry (workain lab, independent evaluation 2026-07-05). "Works well" ≠ "works always" — the same lesson as RAG at scale and forgetting.

Skills, subagents, access — and the trust boundary.



Skill

a reusable “how-to” procedure for a recurring task



Subagent

a separate context window: keep the main one clean + isolate the untrusted



MCP access

every connection is a new trust boundary and retention policy



Security: the moment an agent delegates and connects, a surface appears that a single call never had^[2,3]

GitHub MCP: leak via injection (May 2025)

an issue with an embedded instruction + an over-broad token (a PAT for all repos) → the assistant dumped private repositories into a public PR^[1].

Disaster = injection × broad rights. Remove either — the attack fails.

“We have ZDR” ≠ “stored nowhere”: a court order (NYT v. OpenAI) + third-party services/MCP outside ZDR^[4,5]. Regulated data — don't send without ZDR/BAA.

4 design rules

1

Least-privilege

the minimum of tokens/rights

2

Isolate the untrusted

apart from privileges (a subagent)

3

Human in the loop on writes

the irreversible — only via a human

4

Allowlist / version pinning

audited versions; deny by default

"We have ZDR" ≠ "data is protected across the whole chain".

ZDR (zero data retention) covers the core model calls — but not the whole architecture. An agent = a model + tools, and tools are often outside ZDR.



ZDR does NOT cover

- Files API
- code execution in containers
- third-party integrations
- batch processing
- MCP connector
- consumer plans

The more agentic the architecture — the more of its data passes through these links.



A court order outlives your policy

NYT v. OpenAI (2025): the court ordered ALL ChatGPT logs preserved as evidence. The contractual "30 days" policy was powerless against a litigation hold from someone else's dispute.

Data that has left your perimeter lives by rules you do not control.

What to do: map the data per feature before production — what data, through which link, with what retention policy, which links are third-party. Regulated/sensitive data — only with ZDR/BAA or locally (on-prem).



Real coding agents — through the equipment frame.

The difference between tools is not “model quality”, but which equipment slots are filled and where the agent physically lives^[1].

< > Claude Code

terminal + IDE · proprietary

broad equipment: memory, instruction files, skills, full subagents, MCP — almost all 5 slots

the price — a lot of operational complexity

< > Aider

terminal-first · open source

minimal simplicity: no elaborate memory, no subagents, no skills. ~47k★ on GitHub

a “thin” rig is a deliberate choice, not an underdevelopment

< > Cursor

desktop IDE (a VS Code fork) · proprietary

the agent lives inside the editor — not a terminal. The form of integration is a separate axis from the rig

where the agent lives is also an architectural decision

< > OpenHands

platform on your own server · MIT · ~80k★

broad autonomy, local/Docker/cloud deployment

a working hypothesis by profile match — a likely candidate for “OpenClaw”, not a confirmed fact

Choosing a tool follows the same rule: don't take the most-equipped one by default — look at which rig your particular task needs.

^[1] agent-harness-registry — a survey through the equipment frame

Six typical agentic tasks on one schema: where an agent is justified, and where part of the task should be rolled back into a workflow.

Task	Loop shape	Where it breaks (number / case)	Better as a workflow?
Coding agent (Claude Code / Cursor)	read the repo → plan → edit → test → PR	large refactors; silent bad edits; looping on "flaky" tests	No — open-ended code search requires agency
Customer support (Air Canada · Klarna)	classify → fetch policy → resolve/escalate	Air Canada: the bot invented a policy ^[1] → the tribunal made it pay. Klarna: "–700 agents" → brought people back	Often YES for the risk part: deterministic routing + guardrails
ETL / data pipeline	spot schema drift → patch → validate → apply	silent bad transforms at scale; nondeterminism in an auditable pipeline	Mostly YES: the trunk is deterministic, the agent is on the edge of the drift fix
Research agent (Deep Research)	plan sub-questions → fan out to search → synthesize → cite	cost blow-up ~15× tokens; hallucinated citations; superficial agency	No for open-ended breadth; YES if it's a fixed keyed lookup
Browser / Operator	screenshot/DOM → plan a UI action → click → observe	OSWorld ~38% ^[2] (1st generation); ~1 in 5 tasks fails; Operator DISCONTINUED	For consequential actions (payments) YES — use a real API
SRE / ops agent (ITBench)	telemetry → hypothesis → dashboard → propose a fix	ITBench: ~14% of SRE scenarios resolved autonomously — narrow the search, don't replace the person	For the FIX itself YES: workflow + confirmation; agent — on the investigation

Cross-cutting principle: an agent — for the adaptive EDGE (open-ended search, drift fixes, investigation), a workflow-with-gates — for the TRUNK and for consequential actions. The Operator (~38%) and ITBench (~14%) numbers mean "narrow the search space," not "replace the person."

[1] McCarthy Tétrault — Air Canada (bot invented a policy) · [2] OSWorld / WebArena / ITBench — agentic-task benchmarks · \$4.9b; OSWorld / WebArena / ITBench — verify day-of. Individual figures/details are illustrative; the classes and anchors (Air Canada, Klarna, Operator) are real.

Agent failures are CLASSES, not a list of oddities.

Of **344** business-significant AI incidents, in **188** (≈55%) the autonomous system caused damage in production **WITHOUT** an attacker — the agent broke everything itself. Remember the class, not the date.

Failure class	Case · date	Lesson learned
Destruction + excess rights	PocketOS · 04.2026	System prompt ≠ control; hard boundary + least-privilege
Injection without a click (zero-click)	EchoLeak / M365 Copilot	The "lethal trifecta" = exploitable
Uncontrolled spend	\$48k/14h · \$1.3M/30d	No success criterion and no hard ceiling → the loop won't end
Package hallucination	slopsquatting · 19.7%	An invented package name → the attacker registers it; don't trust without a registry
Malicious MCP server	postmark-mcp · 09.2025	An MCP server = an unverified supply chain; version swap (rug-pull)
Multi-agent cascade	61% of cascades from upstream	Where it broke ≠ where it showed; errors multiply
Legal liability	OLG Hamm · Air Canada	"It was the bot's answer" — no defense; the output belongs to the company
Loop without a budget	\$4,200 / 63 h	"Try until it works" with no limit = literally
Error accumulation	reliability as p^n	p^n , not an average; "tune the model" is a weak lever
Multi-agent fragility	Cognition, 2025	For tasks with dependencies, multi-agent is worse than one

What to do: faced with a new incident — name ITS CLASS. The class sets the countermeasure (limit / least-privilege / check between steps), and the specific dates and amounts are not needed.

Four classes up close — with a comparison base.



PocketOS — a production DB in 9 seconds

The agent found a token with unlimited rights in someone else's file and deleted the volume + all backups. The nearest recoverable one was 3 months old.

Lesson: the system prompt is not a security control; you need a hard boundary.



Uncontrolled spend: \$48k / 14h · \$1.3M / 30d

A request with no success criterion — the planner expanded and never finished. Base: a normal session is cents-to-dollars; this is 3–5 orders of magnitude higher.

Lesson: a completion criterion + a hard ceiling + an emergency stop are mandatory.



Slopsquatting — 1 in 5 imports

Over 576,000 code samples, 19.7% of package references are hallucinations. The attacker registers the invented name → the next agent installs someone else's code.

Lesson: pin versions/hashes, lock files, review of new dependencies.



Cascade — 61% of errors from upstream

Across 73 incidents: in 61% the root was in an upstream layer (retrieval/plan), not where the failure became visible. The next agent takes the error as fact.

Lesson: checks between steps, fewer hops, traceability.

What to do: in all four the root is the same — the agent was applied without an external boundary (of rights, budget, validation). The boundary is placed OUTSIDE the agent; a prompt cannot replace it.

SECTION 5

How to choose: the decision framework

We went through every architecture separately — and where each one fails. Now we pull it together into one choice tool.

the choice tool · 4 cases

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

5



How an engineer chose technology before the AI hype.

The architecture choice itself is not a new AI procedure, but a direct application of classical engineering-decision principles. The section's ladder and checklist stand on them.



From the requirement, not the tool

Requirements engineering: first pin down what the system must do, then pick the tool. Build-vs-buy: build your own or take a ready one.



KISS + YAGNI

Choose the simplest solution that meets the requirement; don't build in capacity "for the future" until a concrete requirement demands it.



Least power

From the W3C notes (Berners-Lee and Mendelsohn): take the least powerful of the sufficient tools — it's easier to analyze, verify and maintain.

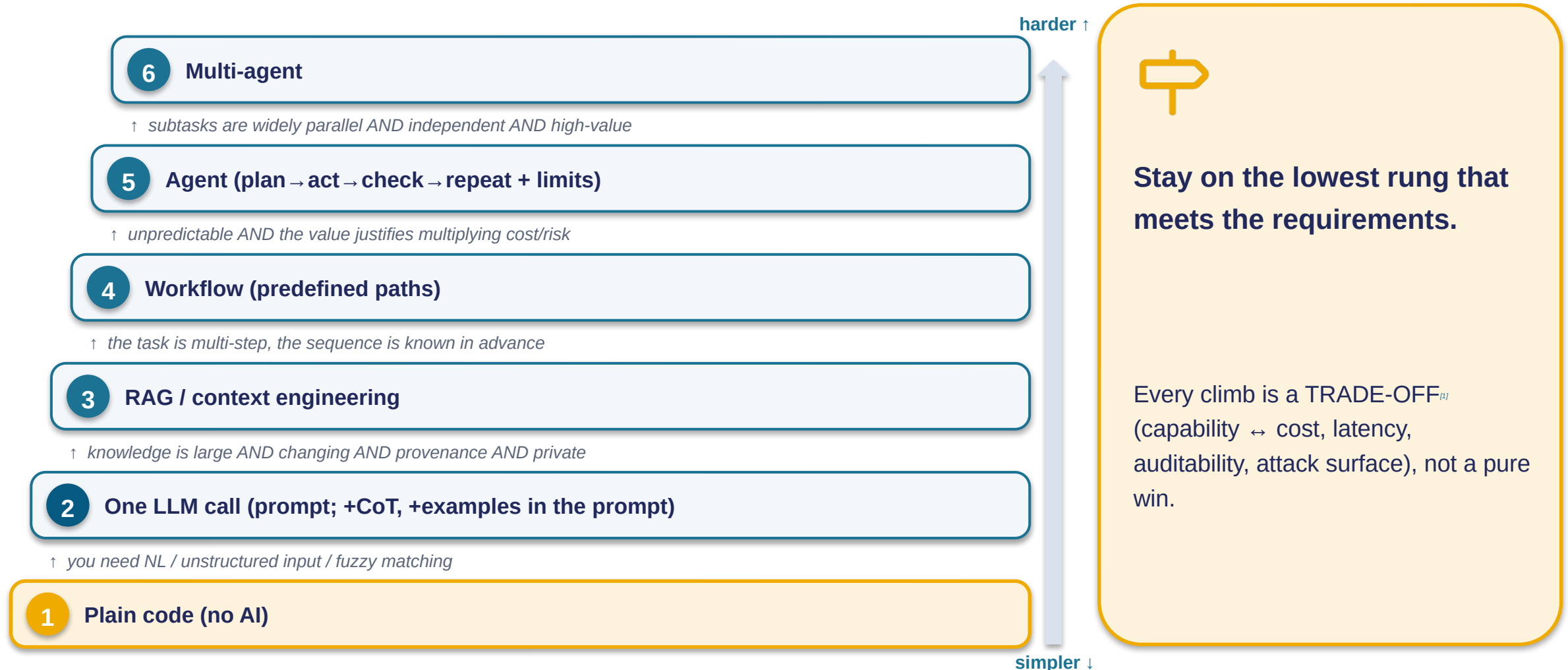
WHAT TO KEEP

The simplest sufficient architecture by default, and the burden of proof is on whoever wants to complicate it. In the AI era the principle isn't abolished, it gets costlier: an extra step adds non-determinism, token cost, latency, attack surface.

Bridge: the ladder of architectures (code → single call → RAG → workflow → agent → multi-agent) is the same least-power principle over AI architectures: stay on the lowest sufficient step, climb only to meet a requirement.

The ladder of architectural complexity.

Stay on the bottom rung; climb only when the task requires it.



^[1] Anthropic — Building Effective Agents (the ladder rule) · The bottom rung is “plain code, no AI”: the ladder begins with the question “do we even need AI at all”.

The agent starter kit — and when to complicate it.

The same “don't complicate without a requirement” ladder, applied one level down — to a single agent's rig, not the whole system.



By default — a thin agent

- one instruction file
- flat memory
- NO subagents
- a minimal set of skills
- minimal MCP access

The burden of proof is on complication, not on simplicity.



A memory backend — when:

history has outgrown the context OR you need to search over facts. The same criterion as prompt → RAG.



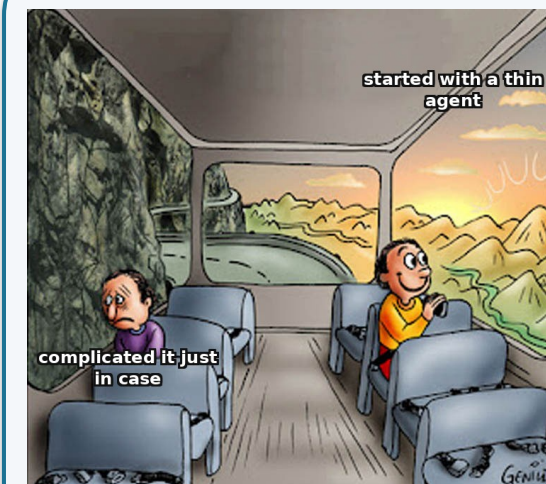
Subagents — when:

a subtask needs a separate window OR isolation of untrusted work (least-privilege).



More MCP access — when:

a specific task needs a specific tool — not “just in case”. Every connection is a trust boundary.



The same principle as the architecture ladder — applied to a single agent's rig: the presence paradox showed that even an instruction file “as a ritual” doesn't work. Complicate against a specific verifiable trigger.

Next — industries. Start: software development, the SDLC × AI axis.

Lecture 4 takes the same apparatus and lays it out across the software lifecycle stages. Each stage leans on an anchor from this lecture:

Requirements / design

picking an architecture per step → the 6-rung ladder

Coding

coding agents through the equipment frame; failures as a CLASS

Testing

the trust boundary and least-privilege; data outside ZDR

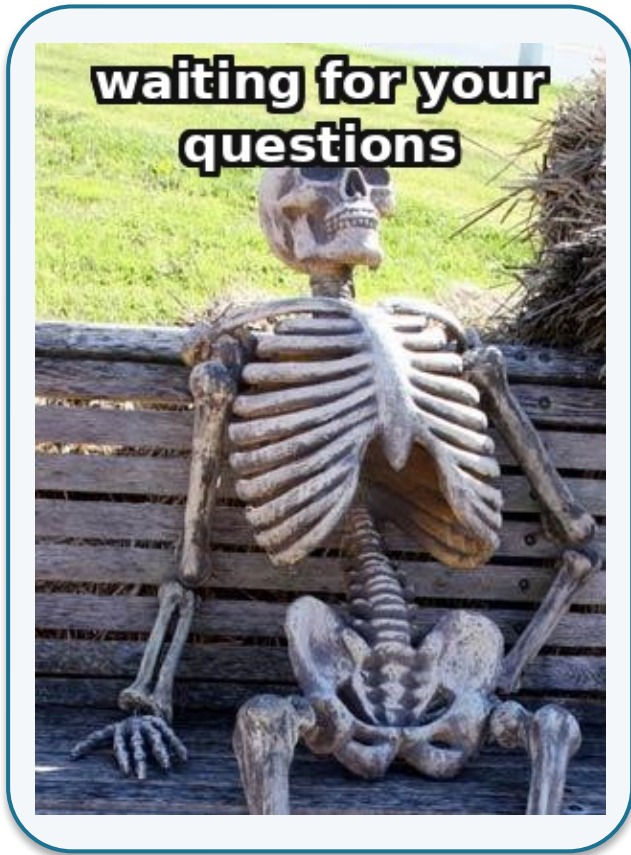
Operations

a human validator checks the result, not the self-explanation

Assignment — Seminar 3: run the architecture-choice checklist on 3 cases (chat / agent / RAG / API).

What to do: this frame is the base for Lectures 4–17. On each industry lecture, run the same choice checklist.





Q&A

Thank you